Lab course Automation and Robotics			
University of Applied Sciences Würzburg-Schweinfurt	Faculty of Electrical Engineering	IMC-5 WS 19/20	Prof. Dr. B. Müller Prof. Dr. J. Hansmann Prof. Dr. T. Kaupp

1 Contents

1 Contents 1
2 Introduction 4
2.1 Preliminary Note 4
2.2 Safety instructions 4
3 Theoretical basics for the control of industrial robots 5
3.1 Definition 5
3.2 Kinematics of an industrial robot (IR) 6
3.3 Coordinate system (CS) 6
3.4 Mathematical Description 9
3.4.1 TCP – Position und Orientation 9
3.4.2 Euler angle 10
3.4.3 Forward transformation 12
3.4.4 Reverse transformation 13
3.5 Motion types of an robot (PTP / LIN / CIRC) 13
3.6 Laboratory equipment 15
3.7 Programming of an industrial robot 16
3.7.1 Basic programming procedure (Online / Offline) 16
3.7.2 Online Programming (Teaching at the Robot) 16
3.7.3 Jogging the robot (teaching) 18
3.7.4 Programming language (KRL) 18
3.7.5 Analog and digital in- and out-puts (including networks) 21
3.7.6 Tools 22
4 Hands-on section 23
4.1 Start up and initialization of the system 23
4.2 First steps 23
4.3 Online-Programming: Pick&Place (VT1) 24

4.4 Offline-Programming: Pick&Place (VT2) 25
4.5 Offline-programming: complex path 27
4.5.1 Constant orientation of the tool (VT3) 27
4.5.2 Variable orientation of the tool (VT4) 29
4.5.3 Programming in the tool-based coordinate system (VT5) 30

Lab course Automation and Robotics			
University of Applied Sciences Würzburg-Schweinfurt	Faculty of Electrical Engineering	IMC-5 WS 19/20	Prof. Dr. B. Müller Prof. Dr. J. Hansmann Prof. Dr. Tobias Kaupp

Experiment preparation
Experiment R: industrial robots programming

The signature confirms that I have taken note of the extended safety regulations in this document.

Course date: _____

Group: _____

Group member:

Name _____ signature _____

Name _____ signature _____

Name _____ signature _____

Name _____ signature _____

Your preparation must include the following steps:

3.7.3 Jogging the robot (teaching)
Read the needed information in the description

4.3 Online-Programming: Pick&Place (VT1)
Sketch with all needed points and create the code

4.4 Offline-Programming: Pick&Place (VT2)
Generated code

4.5.1 Constant orientation of the tool (VT3)
Generated code

4.5.2 Variable orientation of the tool (VT4)
Generated code

4.5.3 Programming of in a tool-based coordinate system
Generated code

2 Introduction

2.1 Preliminary Note

In this laboratory experiment, you will make your first experiences with a 6-axis robot. Therfor, we will use a „Kuka Robot KR AGILUS sixx“ (kr6r900). In order to cope with this in the limited lab time, sufficient preparation is necessary.

The operation of a robot is complex, and cannot be described in all details in the provided description of the experiment. Therefore, this description refers in certain parts to the original documentation from KUKA (KSS_83_END_en.pdf). The KUKA original manual is available as hardcopy in the laboratory at the date of the experiment (you do not need to print it!). Please read the respective chapters and make the required preparations.

In this description, necessary preparations are labeled as **Vorbereitung V!** (german for preparation).

2.2 Safety instructions

When operating industrial robots, strict safety regulations must always be complied with in order to avoid accidents and injuries to people as well as damage to equipment and installations.

During the experiment, the robot is operated exclusively in Teach – Mode (T1). For this reason, operation without safety cage is permitted. Nevertheless, this requires increased precautionary measures.

In general, you have to comply with the following safety measures:



- Under no circumstances, switch on the system without a supervisor and strictly follow the instructions of the supervisor.
- When the robot is moving, everyone (except the operator) must be outside the working area of the robot.
- One group member (not the operator!) or the supervisor always keeps the emergency stop button in his hand. If it becomes necessary, he/she has to use the emergency stop as quickly as possible.
- Special care must be taken when using the gripper. You should reach near the gripping jaws under no circumstances. Moreover, do not try to adjust parts that are manually, while the robot is running. There is a high risk of injury. The pneumatic gripper (operating at about 4 bar / 58 PSI) cannot be released by hand. Only the operator is allowed to correct the position of a part in the gripper after he/she stopped the robot!
- Only the operator is allowed to stay inside the operating area of the robot.

3 Theoretical basics for the control of industrial robots

3.1 Definition

VDI 2860
Basically translated from Weik M.H. (2000) manipulating industrial robot. In: Computer Science and Communications Dictionary. Springer, Boston, MA

„A robot that (a) is automatically controlled, (b) is programmable and reprogrammable, (c) serves a variety of purposes, (d) performs manipulative actions, (e) has several degrees of freedom, usually including displacement, rotation, and articulation, (f) may be either fixed or mobile, and (g) is used in industrial automation applications, such as materials handling, machine control, fabrication, and assembly."

In the majority of applications, industrial robots (IR) are currently used in stationary applications (see Figure 3-1, A). Nevertheless, stationary applications also include robots that are mounted on a linear moving unit (7th axis).

Mobile applications (keyword: autonomous robots) are still mostly subject of research, even though there are already some mobile applications today (see Figure 3-1, B). In the shown examples, the robot can maneuver autonomously through a factory. Other "simple" applications are robotic lawn mowers or robot vacuum cleaners.

A B

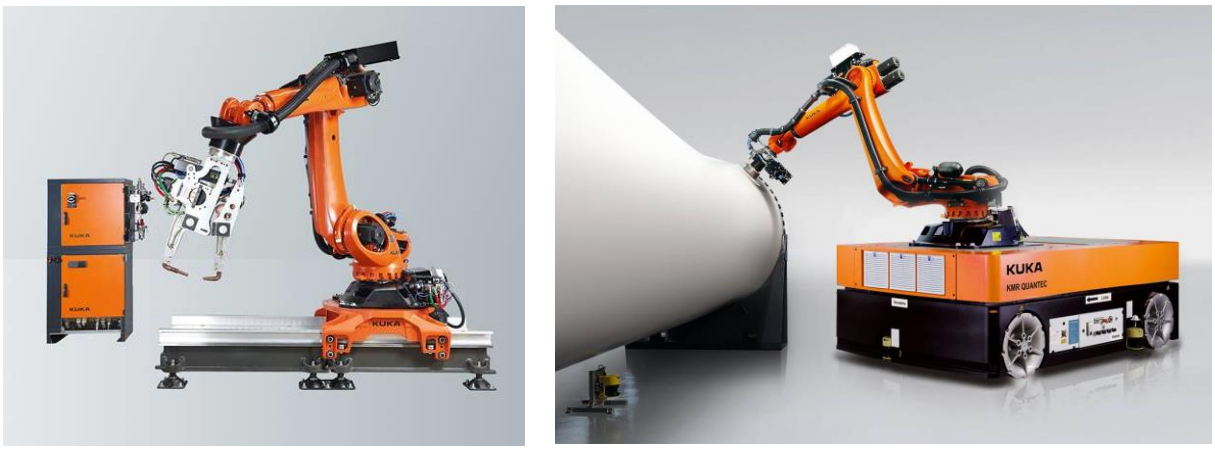


Figure 3-1: (A) „stationary" Robot on a linear unit und mobile Robot

3.2 Kinematics of an industrial robot (IR)

Industrial robots are handling machines with several axes/joints. The arrangement of the axes and joints is called the kinematics of the industrial robot.

A distinction is made between parallel and serial kinematics. With parallel kinematics, several elements are linked to a single joint (see **Fehler! Verweisquelle konnte nicht gefunden werden.**, A). In serial kinematics, the axes and joints are arranged one after the other in a chain (see **Fehler! Verweisquelle konnte nicht gefunden werden.**, B).

A



B



Figure 3-2: (A) parallel kinematics and (B) serial kinematics.

3.3 Coordinate system (CS)

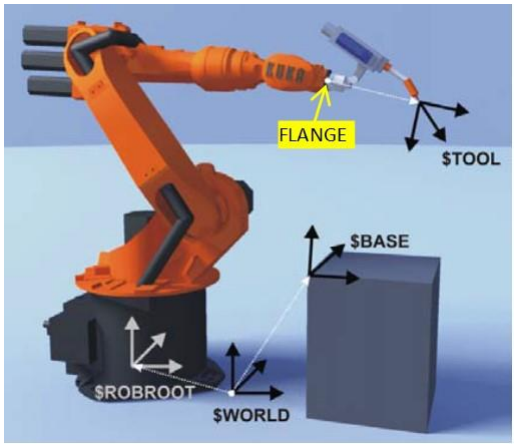
Cartesian coordinates are used for the mathematical description of the position of the tool (TOOL or TCP - Tool Center Point). They are also used for all other relevant robot elements and objects in working area.

In addition to following a defined path, the primary task of a robot system is the exact positioning of the tool center point (TCP). This is the effective point of the tool, which is positioned by the robot. An essential task in the programming of an IR is the definition of the position of the TCP, in terms of both position and orientation.

KUKA robots primarily require the following coordinate systems (Figure 3-3):

- $WORLD
- $ROBROOT
- $BASE
- $FLANGE
- $TOOL

A



B

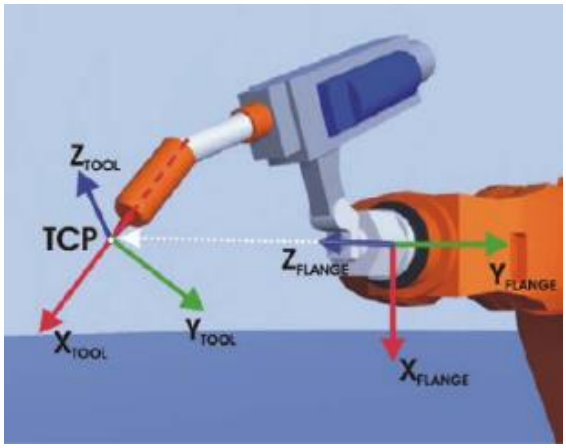


Figure 3-3: (A and B) different reference and tool coordinate systems

The coordinate system $ROBROOT (in the teach-pendant called $BASE0) is always in the robot base and by default also the $WORLD CS. For later practical use (online programming), BASE and TOOL are the most important coordinate systems.

The $WORLD coordinate system is usually only changed when using an additional robot console or a linear axis (see Figure 3-2). When using a linear axis, the $WORLD coordinate system, the coordinate system of the linear axis ($ERSYWORLD) and the coordinate system of the robot ($ROBROOT_C) determine the exact position of the robot on the linear axis.

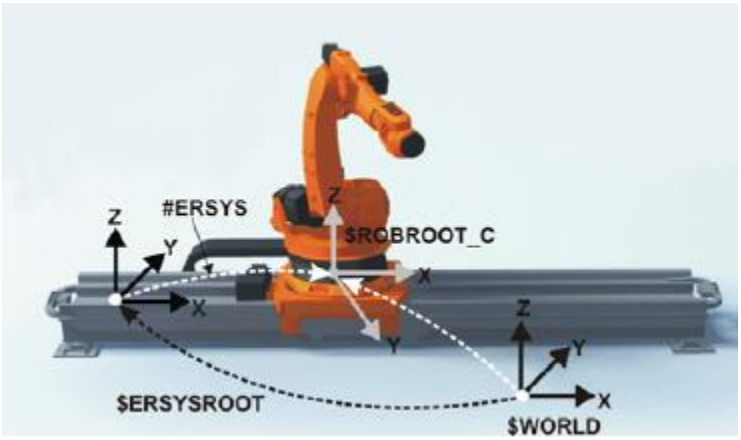


Figure 3-4: coordinate systems using a linear unit

The $BASE coordinate system is normally adapted to the fixture, where the tasks of the robot is performed or in the design origin of the part the task is performed on, e.g. in vehicle construction on the vehicle coordinate system - fzg0 or car0.

The $FLANGE coordinate system is used to define the tool-specific $TOOL coordinate system. In delivery conditions, the $TOOL coordinate system is located in the $FLANGE coordinate system.

In the field, the $BASE0 is used for pure handling processes and only an additional $BASE for fixtures is used for joining processes (welding, riveting, clinching, bolts ...). The additional $BASE is used to increase the accuracy and to compensate deviations of the installation, on the site. Accordingly, programs to control the motion of the robot and that are generated in a computer can be loaded directly into the robot and tested. For this purpose, the fixtures in the factory must be measured beforehand.

In the lab course, you can use the $BASE0 and the corresponding $TOOL for the respective tool. Here, $TOOL1 (electric gripper) or $TOOL2 (ballpoint pen) or optionally $TOOL3 (pneumatic gripper).

3.4 Mathematical Description

3.4.1 TCP – Position und Orientation

To describe the position of the TCP (in relation to the reference system) it is sufficient to define a respective vector ${}^{K_0}\vec{P}_T$. The index K_0 indicates that the components of the vector were specified in relation to the base system K_0.

$$ {}^{K_0}\vec{P}_T = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix} $$

To specify the TCP orientation, it is necessary to define three unit vectors that establish the base of the coordinate system K_w. Often, the following matrix representation is used:

$$ {}^{K_T}_{K_0}R = \begin{bmatrix} \vec{e}_x & \vec{e}_y & \vec{e}_z \end{bmatrix} = \begin{bmatrix} e_{xx} & e_{yx} & e_{zx} \\ e_{xy} & e_{yy} & e_{zy} \\ e_{xz} & e_{yz} & e_{zz} \end{bmatrix} $$

This matrix describes the position of the unit vectors of the coordinate system K_T in relation to the coordinate system K_0. This matrix is called the rotation matrix from K_T to K_0 and defines the orientation of a coordinate system that is "attached" to the TCP in relation to a reference system K_0 and forms the basis for calculations of the robot control.

The exact location (position and orientation) of the TCP is defined by the rotation matrix R and the displacement vector $\overrightarrow{P_T}$. In order to obtain a mathematically closed form, both components are combined in the so-called homogeneous matrix T (also called frame).

$$ {}^{K_w}_{K_0}T = \begin{bmatrix} \vec{e}_x & \vec{e}_y & \vec{e}_z & \vec{p}_E \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} e_{xx} & e_{yx} & e_{zx} & p_x \\ e_{xy} & e_{yy} & e_{zy} & p_y \\ e_{xz} & e_{yz} & e_{zz} & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

This frame unambiguously determines a target position of a robot. The definition of a target point is therefore the same as the definition of the frames above.

3.4.2 Euler angle

Beside the above-mentioned frame, there is another common way to define the target position, the Euler angle. Euler angles describe the rotation of any cartesian coordinate system using three angles. The rotations around these three angles always take place sequentially and refer to either fixed or moved rotation axes. The choice of the order of the rotations results in different conventions.

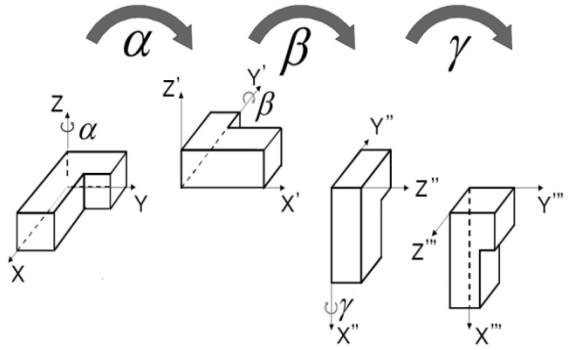


Figure 3-5: z-y'-x''-convention

The *z-y'-x"* convention specifies that all three rotations are performed in relation to the moving object coordinate system. The initially unrotated object-based coordinate system, is rotated around its own *z*-axis by the angle *α*. Then, a rotation around its co-rotated *y´*-axis around the angle *β*. Finally a rotation around the co-rotated *x´´*-axis with the angle *γ*. After these three rotations, the object has the orientation specified by the Euler angle. The rotation matrix is determined from the Euler angle as follows (*z-y'-x''*-convention):

$$R = \begin{pmatrix} \cos\alpha\cos\beta & \cos\alpha\sin\beta\sin\gamma - \sin\alpha\cos\gamma & \cos\alpha\sin\beta\cos\gamma + \sin\alpha\sin\gamma \\ \sin\alpha\cos\beta & \sin\alpha\sin\beta\sin\gamma + \cos\alpha\cos\gamma & \sin\alpha\sin\beta\cos\gamma - \cos\alpha\sin\gamma \\ -\sin\beta & \cos\beta\sin\gamma & \cos\beta\cos\gamma \end{pmatrix}$$

Inversely, the Euler angles can be obtained from the rotation matrix with the following equations:

$$R = \begin{pmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{pmatrix}$$

$$\beta = Atan2\left(-r_{31}, \sqrt{r_{11}^2 + r_{21}^2}\right)$$

$$\alpha = Atan2\left(\frac{r_{21}}{\cos\beta}, \frac{r_{11}}{\cos\beta}\right)$$

$$\gamma = Atan2\left(\frac{r_{32}}{\cos\beta}, \frac{r_{33}}{\cos\beta}\right)$$

falls $\beta = 90{,}0°$

$$\beta = 90{,}0°$$
$$\alpha = 0{,}0°$$
$$\gamma = Atan2(r_{12}, r_{22})$$

falls $\beta = -90{,}0°$

$$\beta = -90{,}0°$$
$$\alpha = 0{,}0°$$
$$\gamma = -Atan2(r_{12}, r_{22})$$

While the rotations around the Euler angle refer to the rotating system, there is also the possibility that the rotations always take place around the axes of the fixed reference system. It can be shown that a rotation sequence *α(z) β(y') γ(x'')* in the rotating system is equivalent to a rotation sequence *γ(x) β(y) α(z)* around the axes of the fixed reference system.

The second convention is used for the definition of positions used by Kuka (Kuka Robot Language). A position *P* is thus defined by three coordinates *X, Y, Z* (position vector from the origin of the reference system to the new coordinate origin) and three angles *A, B, C* (fixed frame, rotations around axes *Z, Y, X* of the reference system).

<u>Attention:</u> For Kuka the definition is A (rotation by z), B (rotation by y), C (rotation by x).
P = (X, Y, Z, A, B, C)

It is important to understand that any such position information always represents a complete coordinate system definition and can therefore always be converted into a homogeneous matrix with the above equation.

Such a (homogeneous) matrix (no matter in which format) can be used in several ways:

- To define a position in the working area in relation to another coordinate system (this is used in robot programming).
- To convert a position given in one coordinate system into another coordinate system. This is done in the robot controller (e.g. forward transformation, see next chapter), but also for offline programming if relative positioning is used (last tasks).
- In graphic data processing for the execution of rotations of bodies in the space (from there comes the name Frame).

In the last two cases, the conversion is done by multiplying two homogeneous matrices. The result is again a homogeneous matrix, a new position.

3.4.3 Forward transformation

As already mentioned, industrial robots are a chain of kinematic elements. Each element is connected to the following element by a joint. The position of each individual joint in space is described by a homogeneous matrix, which refers to the coordinate system previously located in the chain. The last element in the kinematic chain is the tool (TCP).

For example, if a robot has six joints, a homogeneous 4x4-transformation matrix is created for each joint according to the scheme shown in Figure 3-6. By multiplying the six matrixes, the transformation rule is obtained to transfer points from the tool coordinate system into the world coordinate system.

$$T_w = T_{g1} * T_{g2} * T_{g3} * T_{g4} * T_{g5} * T_{g6}$$

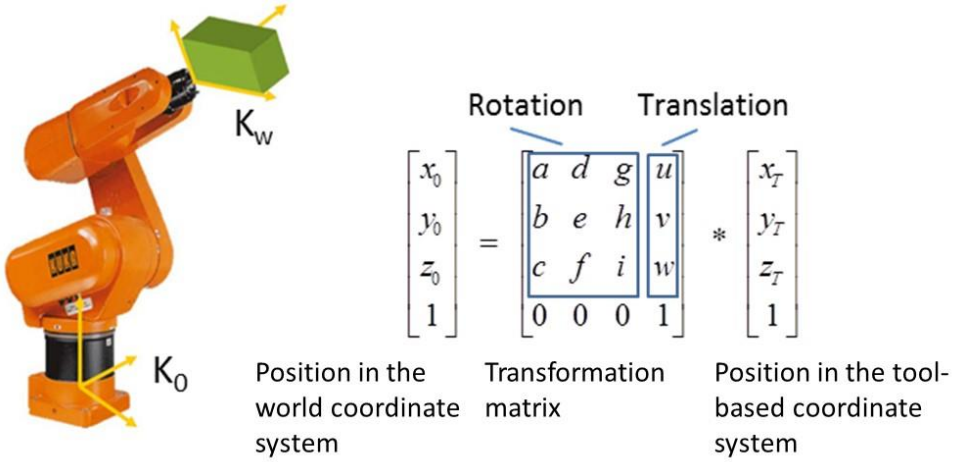


Figure 3-6: transformation

With special design rules for the individual joint matrixes, e.g. the Denavit - Hartenberg - Convention, the matrixes can be setup in such a way, that only the current joint position, for example the joint angle of a rotary joint, changes the content of the matrix. The remaining parameters of the matrix are fix and depend on the geometric structure of the robot.

With the Denavit-Hartenberg-Convention, it is possible to create a matrix for each element of a robot, which indicates the position of the element in relation to its upstream arm part and depending on the current position of the connecting joint.

Such a matrix *T* has the following form:

$$T = \begin{pmatrix} \cos\theta_i & -\sin\theta_i\cos\alpha_i & \sin\theta_i\sin\alpha_i & a_i\cos\theta_i \\ \sin\theta_i & \cos\theta_i\cos\alpha_i & -\cos\theta_i\sin\alpha_i & a_i\sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

If you know all joint angles Θ_1 to Θ_6, the position of the TCP in the reference system can be determined by multiplying the corresponding six matrixes. This process is called forward transformation or forward kinematics. In short, the position of the TCP is determined from the known twisting angles of the robot.

3.4.4 Reverse transformation

When programming an industrial robot, the operator specifies the desired position in the working area (homogeneous matrix, Euler angle, fixed frame angle). To be able to control the robot, the robot controller needs the current position of all the joints. Determining the joint angles to a desired TCP position (and orientation) is called reverse transformation or inverse kinematics. For this important transformation step, depending on the complexity of the robot, analytical methods exist for the simpler kinematics. Numerical methods are available for the more complex kinematics (with more than six joints). One of the problems is that a specific TCP position can possibly be realized by several robot positions (see Figure 3-7). These ambiguous positions sometimes cause problems during programming and even when teaching a robot. Normally, the programmer has to specify, which solution should be used. Positions in which an infinite number of reverse transformation solutions exist (singularity) should be avoided or only approached in a special motion type (PTP, see next chapter).

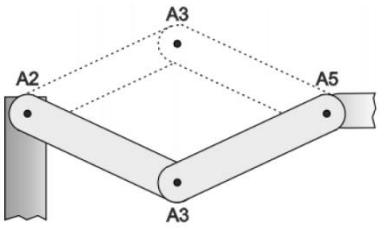


Figure 3-7: example for an ambiguous position

3.5 Motion types of an robot (PTP / LIN / CIRC)

There are several motion types for industrial robots (see also KSS_83_END_en.pdf / Chapter 8, Figure 3-8). One of these motion types is the point-to-point movement (PTP). In a PTP movement, the necessary rotations or translations are determined independently for all joints of a robot. This is done to reach from the initial position to the target position as quickly as possible. In the simplest form, the asynchronous PTP movement, each joint is twisted or shifted to its target position as quickly as possible, while complying with its mechanical and dynamic limitations.

The direction of rotation or translation of the axes remains constant over the movement. The resulting paths of the individual robot elements are difficult to predict for the user. The TCP in particular can describe a movement whose change in position and orientation is neither uniform nor even linear. However, a PTP movement is the fastest possible form of position change. It makes full use of the maximum acceleration and speed values of the robot.

To reduce the mechanical load on the robot, synchronous PTP movement exists. First, a master

axis is determined whose movement takes the longest time. Then, the maximum speed of all other axes are reduced so that all movements end at the same time. The total time of movement is not affected. Nevertheless, all axes whose speed is reduced to facilitate a shorter acceleration and braking phase, have a reduced mechanical load and thus a less abrasion. Alternatively, the maximum acceleration can be reduced, resulting in the same effect.

For many applications of a robot, the defined movement of the TCP on a given path is just as important as the determination of the start and end position. There are motion types, which, in contrast to PTP motion, allow the TCP path to be defined (Figure 3-8). In order to keep the mathematical description of the paths simple, most robot controllers offer the motion on a straight line (LIN) and the motion on a circular path (CIRC). In this motion type, the user defines the start and end position (and also a third point for the circular motion) of the TCP in a Cartesian coordinate system. To specify the position of the TCP, both the position and the orientation must be defined. The orientation is usually given in Euler angles. The robot controller then moves the robot in such a way that the movement of the TCP describes a straight line or circular movement in the work area.

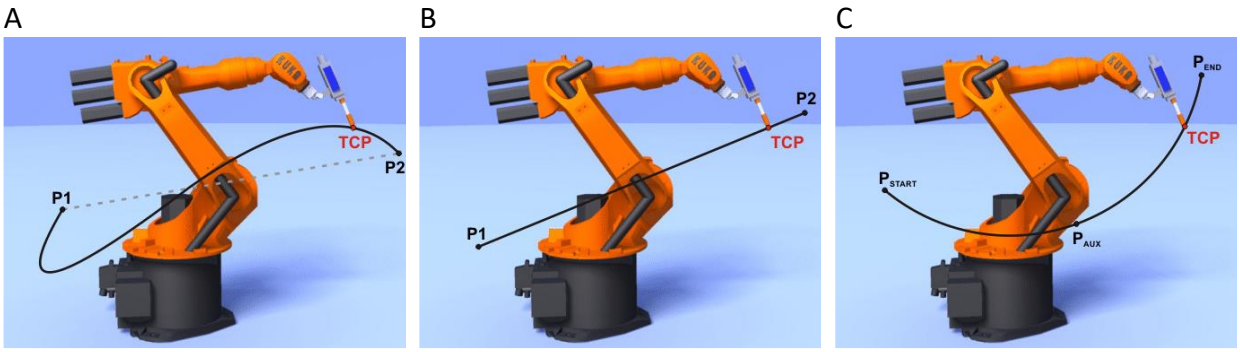
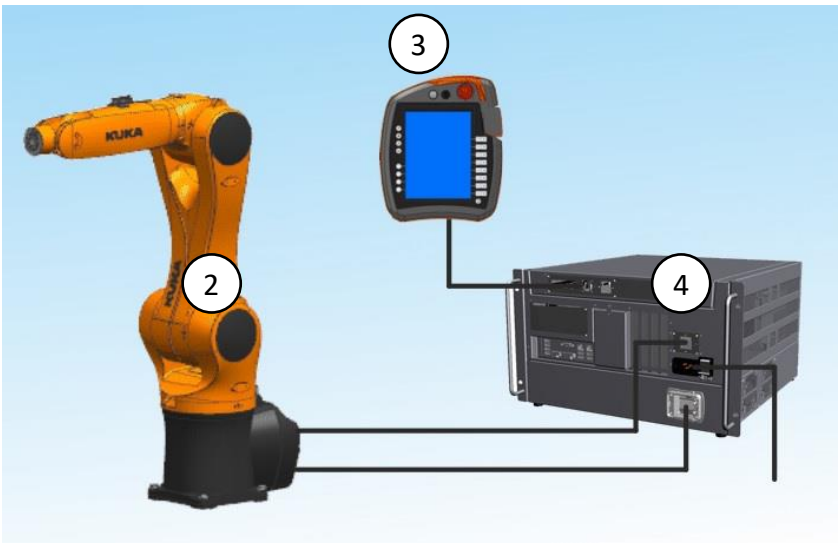


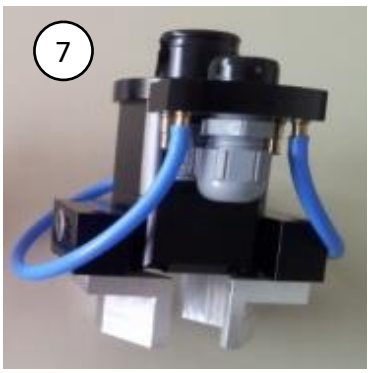
Figure 3-8: (A) PTP-Interpolation, (B) LIN-Interpolation, and (C) CIRC-Interpolation

3.6 Laboratory equipment











1. offline programmer (PC)
2. Kuka Robot (kr6r900)
3. Teach-Pendant (smartPAD)
4. Robot control (KRC4 compact)
5. tool – electric gripper
6. tool – ballpoint
7. tool – pneumatic gripper

Figure 3-9: components of the laboratory equipment

The system consists of the actual robot (kr6r900), the controller (KRC4 compact) and the connected teach pendant (smartPAD) for online programming. In addition, a computer is available for offline programming. Various programs can be used for offline programming.

Primarily Orange-Edit.Free (as free software), Process Simulate and KuKa SimPro with OfficeLite are available in the laboratory. The freeware OrangeEdit.Free is the best option for you at home to get familiar with the Kuka code. There are also tools (gripper and ballpoint) that can be mounted to the robot arm using a fast change system. The pneumatic gripper has compressed air connections for opening and closing the jaws.



Caution! Risk of injury: Never change the tools yourself. The gripper and fast change system is under pressure, you cannot simply open the locking mechanism!

3.7 Programming of an industrial robot

3.7.1 Basic programming procedure (Online / Offline)

Programming a robot requires appropriate planning and a systematic, structured and step-by-step approach. A complex task can be broken down and refined step by step. Alike the development of conventional software, a precise definition of the task is a precondition. In program development, a basic distinction is made between two methods, online and offline programming. The basic procedure is similar in both cases. In this experiment, both methods are used.

With **online programming**, the program is created directly on the robot using the teach pendant. The teach pendant is used to teach the needed points and then to develop the program structure and logic.

For **offline programming**, all this is done on the computer, both the definition of the points and the definition of the program logic. This requires a lot of preparation, especially with regard to the definition of the coordinate systems and the individual positions.

The second part will therefore use a mixed form, namely the definition of the points (or at least parts of them) with the teach pendant and the development of the program logic with the help of the programming software on the computer. Nevertheless, with the planned motions, simulations can be performed, also in a CAD model of the robot environment.

3.7.2 Online Programming (Teaching at the Robot)

At the beginning of every programming task, there is a task description. If there is not already a sketch, one has to be developed. It is important to name all needed positions. If necessary and possible, you also add the orientation of TCP.

Usually, the tool comes in contact with the part during the implementation of a program (handling, welding, …). In order to avoid damage, basic rules must be considered during programming:



The full speed approach to the part or other fix components is only permitted if a defined safety distance is ensured. Below the defined safety distance, only a reduced speed is allowed!



Furthermore, "unknown" programs must always be run slowly! As "unknown" programs all pro-grams are defined, which have not been run on the robot yet! This means that even with small changes in the program code, the program must first be run at reduced speed. After that, the speed be increased up to the desired speed with each further run. In practice (and also in the lab), this means that distances without obstacles may be driven faster, the closer the obstacle the slower to drive!

These approach points must be taken into account during planning as well as during teaching (in part two you will get to know another procedure).

Basic specifications:

Normal speed: PTP VEL=10% (10% of maximum velocity)
Safety distance: 50 mm
Reduced approach speed: LIN VEL=10% or VEL=0.005 m/s (=30 cm/min)

When gripping a part, a sketch might look like shown in Figure 3-10.

From the point P010_start, a PTP approach takes place at 10% speed to point P020, from there an approach to the gripping point on a straight line (linear) at 10% speed. The z-axis of the TCP should be perpendicular to the support surface at P020 (P030_pick und P040).

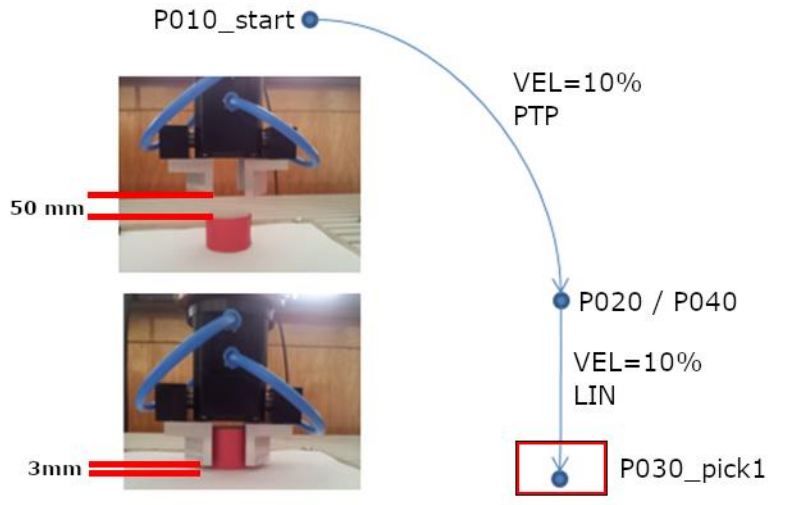


Figure 3-10 program sketch

Such a sketch forms the basis for teaching the individual points and later programming the movement instructions.

3.7.3 Jogging the robot (teaching)

The T1 mode is always used for teaching and the robot only moves at a reduced velocity.



Even in the T1 mode, the robot is still very fast at 100%. Therefore, the speed has to be adjusted accordingly. Never start with more than 10% velocity and then regulate the velocity dynamically depending on the distance to the obstacle!

Two different control elements can be used for jogging the robot:

- Jog keys (A1-A6 or XYZ-ABC)
- Space Mouse (6D-Mouse)

V! Depending on the setting (see manual **KSS_83_END_en.pdf** / chapter 4.13), various operating modes can be set both for the jog keys and for the Space Mouse. For preparation, please read the descriptions in the manual.

Important are also the chapters:

- Velocity - program override (POV) and jog override (HOV) (**KSS_83_END_en.pdf** / Chapter. 4.13.3 and Chapter. 7.5)
- Tool calibration (Tool Data) (**KSS_83_END_en.pdf** / Chapter 5.7.1)
- Base calibration (Base Data) (**KSS_83_END_en.pdf** / Chapter 5.7.2)

3.7.4 Programming language (KRL)

The "KUKA Robot Language" is a proprietary language developed by Kuka. Of course, this language is primarily intended for defining motion instructions, but it can also be used to implement simple calculation algorithms. The necessary operators and control structures are available, but unfortunately, they are by far not as powerful as you are used to from higher languages like C++ or Java.

A Kuka program always comprises two files: "*.src" and "*.dat". The src-file contains the flow logic (when which point) and the dat-file contains the information about the point. It is important to know that there is a Human-Machine Interface (HMI) and an Ascii view. The HMI-view is a simplified view, which is available in OrangeEdit.Free and the teach pendant. The HMI view hides some information completely and is folded additionally. In this document, only the HMI view is shown.

src-file (Figure 3-11**)**
The src-file starts with a header, which contains the definition of the file, a declaration part and the initialization part. This is followed by initialization of the equipment available in the laboratory (pressure for collision detection, electric gripper or pneumatic gripper). This second initialization only ensures that the system is in a defined known state. This is followed by the HOME point of the robot. Programming instructions in the practical course are only created between the two HOME points. Here, these are two PTP points (P0010, P0020) and one LIN point (P0030). Before the definition ends, optional commands to reset the system are possible.

```
DEF test()
Declaration
INI

;-- program
;-- own initialisation of lab equipment
;-- activate pressure
ANOUT CHANNEL_3=0.15
;-- ini electric gripper
wsg50_cmd("home()")
;-- ini pneumatic gripper
; open
$OUT[5] = TRUE
$OUT[6] = FALSE
WAIT Time=1 sec

;-- default home
PTP HOME  Vel= 100 % DEFAULT

;-- own points start
PTP P0010 Vel=100 % PP0010 Tool[1]:PneumoGr Base[0]
PTP P0020 Vel=100 % PP0020 Tool[1]:PneumoGr Base[0]
LIN P0030 Vel=0.005 m/s LP0030 Tool[1]:PneumoGr Base[0]

;-- own points end
;-- default home
PTP HOME  Vel= 100 % DEFAULT
;-- own initialisation of lab equipment
wsg50_cmd("home()")

END
```

Figure 3-11: src-File

dat-file (Figure 3-12)
In the dat-file, the individual definitions are made after the header. In the example shown here, the file is sorted by E6POS, FDAT, PDAT and LDAT. This sorting is not Kuka standard, but can be created in OrangeEdit.Free. The E6POS values are the coordinates of the points (*X, Y, Z, A, B, C*). In addition, an S and a *T* value are defined, that the robot is able to determine the area in which it's located. For example, the information about the axis range of axis 3 is included (see Figure 3-7). If there is further interest, the *S-T* values are described in the system integrator's manual. The FDAT value primarily define the tool and base. The **P**DAT and **L**DAT values define the parameters of the **P**TP and **L**IN points. In the example, the points are approached with fine-mode (APO_DIST 0.0). With approximate positioning, for example with 100 mm, the command would look like (APO_DIST 100, APO_MODE #CDIS).

```
DEFDAT test
EXTERNAL DECLARATIONS

; E6POS declarations
DECL E6POS XP0010={X 700.000,Y -10.0000,Z 300.000,A -180.000,B -0.000,C 180.000,S 2,T 10}
DECL E6POS XP0020={X 700.000,Y -10.0000,Z 180.000,A -180.000,B -0.000,C 180.000,S 2,T 10}
DECL E6POS XP0030={X 700.000,Y -10.0000,Z 155.000,A -180.000,B -0.000,C 180.000,S 2,T 10}

; FDAT declarations
DECL FDAT FP0010={TOOL_NO 1,BASE_NO 0,IPO_FRAME #BASE,POINT2[] " ",TQ_STATE FALSE}
DECL FDAT FP0020={TOOL_NO 1,BASE_NO 0,IPO_FRAME #BASE,POINT2[] " ",TQ_STATE FALSE}
DECL FDAT FP0030={TOOL_NO 1,BASE_NO 0,IPO_FRAME #BASE,POINT2[] " ",TQ_STATE FALSE}

; PDAT declarations
DECL PDAT PPP0010={VEL 100.000,ACC 100.000,APO_DIST 0.0,GEAR_JERK 50.0000}
DECL PDAT PPP0020={VEL 100.000,ACC 100.000,APO_DIST 0.0,GEAR_JERK 50.0000}

; LDAT declarations
DECL LDAT LLP0030={VEL 0.00500000,ACC 100.000,APO_DIST 0.0,APO_FAC 0.0,AXIS_VEL 100.000,
                   AXIS_ACC 100.000,ORI_TYP #VAR,CIRC_TYP #BASE,JERK_FAC 50.0000,
                   GEAR_JERK 50.0000,EXAX_IGN 0}

ENDDAT
```

Figure 3-12: dat-File

The standard commands can be found in the inline form (OrangeEdit.Free or teach pendant).
The following commands are also useful for the course:

;gripper close	Comment, begins with a semicolon";"
$OUT[5] = TURE	digital output (here 5) (state "TRUE" or "FALSE")
$ANOUT[3]=0.15	analog output (here 3) (status: 0,15)
WAIT Time=2 sec	Wait command (program waits 2 seconds)
PTP_REL {X 0,Y 0,Z 10}	relative PTP-Movement (e.g. z=10mm), also Rotation possible (A,B,C)
LIN_REL {X 0,Y 0,Z 10}	relative LIN-Bewegung (e.g. z=10mm), also Rotation possible (A,B,C)

Furthermore, you need the existing program "wsg50_cmd" to control the electric gripper. You can call this in your program as shown below:

wsg50_cmd("PARAMETER")

Example: wsg50_cmd("home()")

The required or most important “PARAMETER” are:

home()	Initialization (must be done at the beginning!)
move(x)	approach certain position (x in mm opening stroke)
grip()	Gripper close
release()	Gripper open (has to follow after grip command!)

A more detailed list can be found in the documentation for the gripper. (**wsg_gcl_reference_manual-4-0-0.pdf**).

3.7.5 Analog and digital in- and out-puts (including networks)

The robot has various analog and digital inputs and outputs.

collision detection

For an optional collision detection, the KUKA has a pressure control valve, which can be controlled via the analog output [3]. Depending on the set pressure, the digital input [1] triggers the gripper at different loads. This can be used to trigger an emergency stop.

The following values should be set by default:
$ANOUT[3]=0.15

This can be done manually or in the program code. Please note that the value in the program code must be set differently than in the Inputs/Outputs submenu.

Pneumatic gripper

For an optional use of the pneumatic gripper, the tool is connected to digital outputs of the robot. A valve to introduce or release compressed air for the pneumatic gripper is driven by the outputs [5] and [6]. With [5] for opening and [6] for closing the gripper. Please note that when controlling both outputs, the gripper will neither open nor close!

```
;gripper open
$OUT[6] = FALSE
$OUT[5] = TRUE

;gripper close
$OUT[5] = FALSE
$OUT[6] = TRUE
```

It should also be noted that the robot does not know whether the gripper is open or closed. You should always add an appropriate waiting time into your program when changing the state of the gripper. This can be done, for example, as follows:

```
WAIT Time=1 sec
```

To note, if you compare this with the subprogram for the electric gripper, it always waits for successful execution. Therefore, no additional waiting time is necessary for the electric gripper.

3.7.6 Tools

In the field, a tool is mounted on the flange of the robot. This tool is normally also mounted during teaching and must be defined in order to avoid incorrect teach positions (and to make the tool exchangeable afterwards). The tool can be defined by different methods. The simplest is the numeric input (KSS_83_END_en.pdf / Chapter 5.7.1). The existing tools have already been created in the lab course.

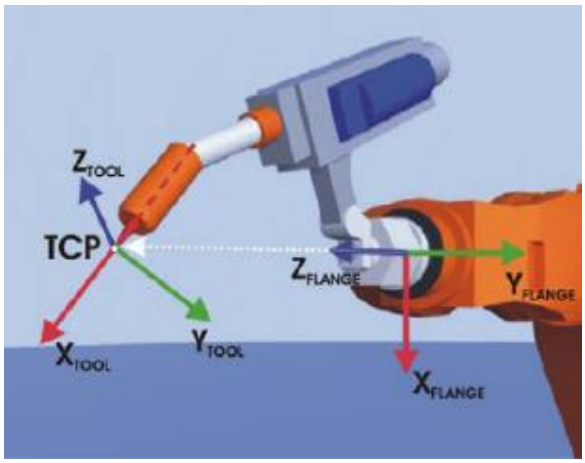


Figure 3-13: Tool-frame

The tool is defined by 6-values and defined relative to the FLANGE (not the BASE).

Pattern: {X VALUE, Y VALUE, Z VALUE, A VALUE, B VALUE, C VALUE}

X, Y, Z	Coordinate system
A	Rotation about the Z-axis
B	Rotation about the Y-axis
C	Rotation about the X-axis

Tool data:

Tool[1]	Pneumatic gripper	{X 0,Y 0,Z 140,A 47,B 0,C 0}
Tool[2]	Electric gripper	{X 0,Y 0,Z 195,A 47,B 0,C 0} (not used this year)
Tool[3]	Ballpoint	{X -39,Y -10,Z 257,A 0,B 0,C -165}

Special care must be taken when handling tools. Incorrect tool data (or tools) are a major cause of collisions. It is essential to make sure that

- The correct data of the mounted tool has been set during teaching
- The tool has been selected correctly
- In automatic mode, the correct tool is mounted
- The gripper is open when you want to grip a part

4 Hands-on section

4.1 Start up and initialization of the system



CAUTION: Do not use the system without a supervisor!
The robot must be in T1 mode. If this is not the case, the robot is not allowed to be operated! You can check and adjust this using the connection manager (key switch).

- The system is switched on with the main switch on the power distribution strip and then with the main switch on the control unit (KRC4 compact).
- Check if the required operating pressure is set to 4 bar. Only check, do not adjust anything! If the pressure is incorrect, report this immediately to the supervisor.
- When the control has been started up, the teach pendant can be used.

4.2 First steps



CAUTION: If one of you moves the robot, all others must maintain a sufficient safety distance. One of the other group members always has the Emergency Stop button in the hand, which has to be pushed immediately in case of danger. A single person alone is not allowed to operate the robot!



CAUTION: The blocks may only be moved or positioned by the person who has the teach pendant. Never let a third person move the blocks. Attention, there is an imminent danger of injury!

Please make sure that everyone has a chance to get a turn and feeling for the device.

<u>Procedure:</u>

- First, determine the position of the axes of the reference system by moving the robot in xyz-mode in the individual axes. Moreover, please check the rotation axes.
- Open and close the gripper (caution: danger of jamming, keep distance from the gripper).
- Try to pick up a part from the table. Move the part upwards a little while holding it and twist the axes. Now try to put the part back in its original position.
- Create a new program and move to the home position. The home position is a fixed position in the robot.



INFO: To change several values, e.g. free typing in command editor you need to log in as “Expert”. If you need this, you can get the credential during the lab.

4.3 Online-Programming: Pick&Place (VT1)

The task is to program a "Pick&Place" operation. All blocks should be moved from the left side of the obstacle to the right side and then back again (see Figure 4-1). The start position on the left side is marked, the target position on the right side not. You are not allowed to remove the obstacle (not even to mark the target position).

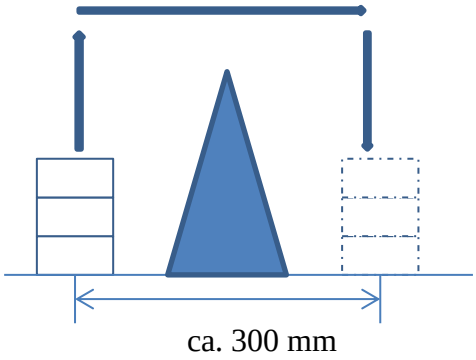


Figure 4-1: Sketch of the task

The movement above the obstacle should be performed in PTP mode. The points, where the velocity is reduced for approaching the block (or the storage position) are each 50 mm in front of the target point. The positioning should be exact.

V! Create a sketch in which you draw and name all the points you need to teach later. In this program, all points will be teached. Try to create the program with Orange-Edit.Free (Only with dummy points, because you will teach them in the lab!).

Design the program with the already defined points in advance. Pay attention to the following:

- the right tool
- the correct velocity
- approximate positioning motions and exact positioning motions
- the correct closing and opening of the gripper
- start and end all programs in Home-Position

Lab procedure:

Comply with safety instructions!

Teach all the points from your sketch with the teach pendant. Please make sure that all team members have the chance to teach the positions. Pay attention to set the correct tool (the electric gripper is tool 1). Name your program according to the structure "degree-programme_Semester_Group_No._VT-No.", e.g. IMC_WS19_G21_V1 (IMC, WS 2019/2020, Group 2.1, VT-Part 1). The teaching procedure is quite intuitive. During the experiment, you will get support if needed. Then, please test the program systematically.

4.4 Offline-Programming: Pick&Place (VT2)

Normally, every single position is taught as it was done in VT1. However, there are applications where a dynamic adjustment of the positions is useful, e.g. for palletizing systems or components with varying lengths. In the case of frequently changed positions, you can also shift the base. Its primary advantage is that points or parts of the path can easily be moved without teaching all points again. With the use of simulation software, such as Process Simulate, and an existing simulation model of the plant, it is sometimes easier to move and test the points or paths completely on a computer.

With this information, the program should be created in a different way. It is possible to calculate all other points (even the bypass point and the points on the other side of the obstacle). All you have to do now is to make the required displacement in the direction of the corresponding axis of the reference system. However, for time reasons, only the pick and deposit positions are to be created as parameter here. Each requires a point to pass the obstacle (yellow) and the two approach points (red) of the most upper part (with 50 mm safety distance). Actually, the path to avoid the obstacles is of course very simple. In real applications, it can be much more complicated. This means that you only need the points shown in Figure 4-2.

V! Refine your program from VT1 in such a way that you now use this programming technique (Also only with dummy points, because you will already teach them for VT1 in the lab and can then work with this points).

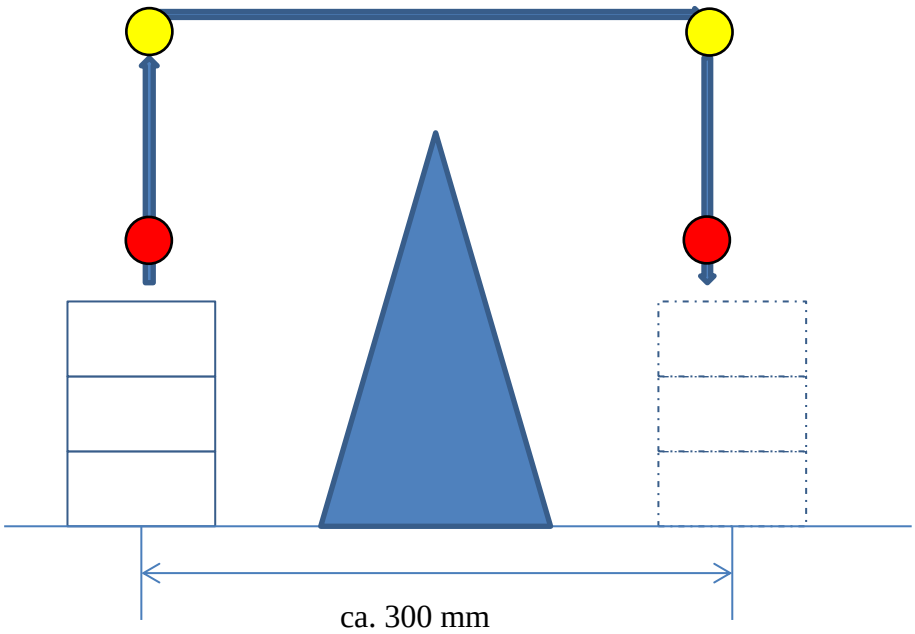


Figure 4-2: sketch of needed points

Lab procedure:

Export your program for VT1 to a USB memory stick and open it in OrangeEdit.Free. Following, delete all unnecessary commands.



Attention: Do not delete any commands before or after the HOME command! The initialization of the gripper is after the HOME command, but this should be kept also. Save the program after which you have implemented your logic under the required file name (similar to VT1).

There are several possibilities for this: a) directly in the src file or b) in the dat file. There is also the possibility to specify relative and absolute values for the movements. Figure 4-3 shows you examples for code directly in the src-file.

```
1  DEF test2()
2
3  Declaration
4  ;-- DECL FRAME-Variable
5  DECL FRAME relFrame
6  INI
7
8
9  ;-- ...
10
11 ;-- LIN relative from current position (z=50mm)
12 relFrame = {x 0,y 0,z 50,a 0,b 0,c 0}
13 LIN_REL relFrame
14
15 ;-- PTP relative from current position (z=10mm)
16 PTP_REL {X 0,Y 0,Z 10}
17
18 ;-- PTP to desired Position
19 PTP {X 700.09,Y 245.98,Z 527.01,A -179.998,B 0,C 180}
20
21 ;-- PTP to desired Axis-Values
22 PTP {A1 0,A2 -60,A3 70,A4 0,A5 90,A6 45}
23
24 ;-- ...
25
26 END
```

Figure 4-3: Code-example

As you can find in the code example (Figure 4-3), you can only use variables if you define them first. In the example it's done under Declaration. Here only a FRAME is defined, it has the same structure as the shown tool values in Chapter 3.7.6. There are two basic ways to move a point BASE-MOVE or TOOL-MOVE. As the names provide the Point will be moved in Base or Tool coordinate System. Both movements can be combined ({BASE} : <POINTNAME> : {TOOL})! It can be useful for a movement in Base direction and an additional rotation around the tool-axis (see VT4). Mathematical it is a matrix multiplication, but you have to consider witch came first.

BASE-MOVE

```
;direct in PTP-move
PTP {x 100, y 0, z 0, A 0, B 0, C 0} : P1
;or with variable
moveFrame = {x 100, y 0, z 0, A 0, B 0, C 0} : P1
PTP moveFrame
```

TOOL-MOVE

```
;direct in PTP-move
PTP P1 : {x 100, y 0, z 0, A 0, B 0, C 0}
;or with variable
moveFrame = P1 : {x 100, y 0, z 0, A 0, B 0, C 0}
PTP moveFrame
```

4.5 Offline-programming: complex path

For example, if you want to investigate the path accuracy of a robot, it is helpful "record" this. In the simplest case, the robot can draw its own path. However, this also corresponds an application in vehicle construction in which glue is applied to the surface of a component. In the laboratory, a self-constructed ballpoint is used as a tool. This is then harnessed to draw on a sheet of paper (see Figure 3-9 [6]).

The task now is to use the robot to draw a certain pattern on the paper. The effect of approximate positioning (C_DIS) or exact positioning (Fine) is also to be investigated. In addition, offline programming is further be trained.

4.5.1 Constant orientation of the tool (VT3)

A ballpoint is used to track path. In contrast to a fountain pen, a ballpoint can write in any directions, so it is not necessary to change the orientation of the pen. The task is to draw the shape shown in Figure 4-4.

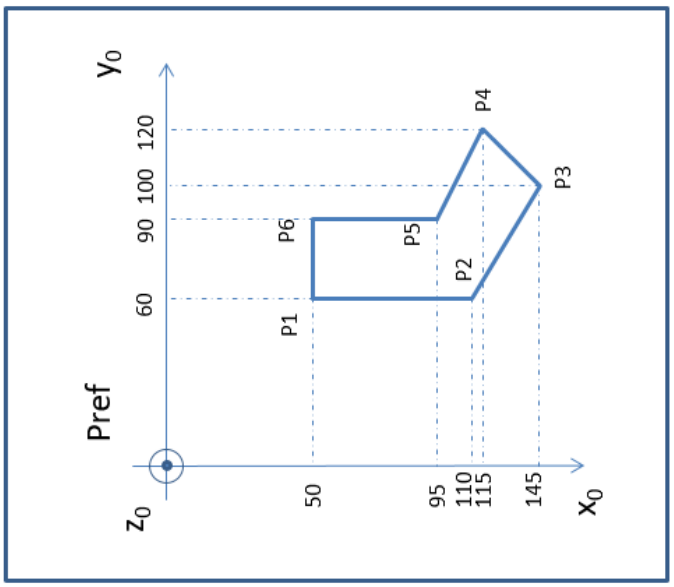


Figure 4-4: sketch for drawing

The reference system for the task description is located at the top left of the drawing sheet. This point must be taught (this is the only position one in this task). In the following (and please in your program) this point is called PRef. Starting from this point, the other positions should be programmed. For this, you can take advantage of the fact that you can simply add two points (this adds the displacement vectors and the torsion angles).

V! Develop a program that moves from the home position to the reference point (distance 15 mm in z-direction) and then draws a small cross at the reference point.

Lab procedure:

Teach your first point on paper (PRef), from this point you can define all other points. If you want to call the coordinate-values for PRef you have to use XPRef (values will be saved with X<name>). Export this program and continue writing the logic on the computer. Make sure that you have set the correct tool. The ballpoint is the tool 3. Implement the following programs one after the other and test them. (Ask the professor or lab staff witch interpolations you should do.)

1.) LIN-Interpolation (Fine)
2.) LIN-Interpolation (C_DIS 100)
3.) LIN-Interpolation (C_DIS 1000)
4.) PTP-Interpolation (Fine)
5.) PTP-Interpolation (C_DIS 100)
6.) PTP-Interpolation (C_DIS 1000)

You could solve the problem as shown:

- Each point is programmed in relation to PRef, which would then lead to such instructions (here shown as PTP, for the x-axis of the coordinate mark):

```
;points for coordinate mark
PRef_x10 = {x 10, y 0, z 0, A 0, B 0, C 0} : XPRef
PRef_10x = {x -10, y 0, z 0, A 0, B 0, C 0} : XPRef

;now drive these points
PTP XPRef
PTP PRef_x10
PTP PRef_10x
PTP XPRef
```

```
;points for coordinate mark
PRef_x10 = XPRef
PRef_x10.x = XPRef.x + 10
PRef_10x = XPRef
PRef_10x.x = XPRef.x - 10
;now drive these points
PTP XPRef
PTP PRef_x10
PTP PRef_10x
PTP XPRef
```

- You program relative to the previous point:

```
;points for coordinate mark
PRef_x10 = {x 10, y 0, z 0, A 0, B 0, C 0} : XPRef
PRef_10x = {x -20, y 0, z 0, A 0, B 0, C 0} : PRef_x10

;now drive these points
PTP XPRef
PTP PRef_x10
PTP PRef_10x
PTP XPRef
```

```
;points for coordinate mark
PRef_x10 = XPRef
PRef_x10.x = XPRef.x + 10
PRef_10x = PRef_x10
PRef_10x.x = XPRef_x10.x - 20
;now drive these points
PTP XPRef
PTP PRef_x10
PTP PRef_10x
PTP XPRef
```



Attention: Make sure that your teached point has the correct tool and base! The defined points will always drive with the last used tool and base if nothing else is defined.

4.5.2 Variable orientation of the tool (VT4)

As already mentioned, with a fountain pen the program above would not work. In this case, you have to make sure that the fountain pen always has the correct orientation to the path. Therefore, you have to turn the tool during the process. For VT4, we assume that the orientation is correct if the x-axis of the tool points in the direction of the movement. Before moving to a certain point, the tool must be rotated to the correct direction. With this type of programming, the positioning must therefore be relative to the taught base point PRef, but the angle must be set separately. Otherwise, the orientation of the tool remains the same as used for teaching the reference point.

The corresponding statements would then look like this:

```
;assume that XPref has the values
XPRef = {x 450, y -60, z 0, A 20, B 0, C 180}
;to correct the A-rotation you have to rotate the gripper back to zero degree.
Pref_new = XPref
Pref_new.A = 0
;now drive these points
PTP XPef
PTP Pef_new
```

You can also use the matrix multiplication ":"-Method (defined in Chapter), but be advised that you don't want to rotate around the Base! If you use it you have to rotate around the Tool! It will be used in VT5.

With SIN(x), COS(x), TAN(x), ACOS(x), ATAN2(x,y), you can calculate any required angles if you have not already done so. (For ASIN(x) you need to use ACOS → ASIN(x) = 90-ACOS(x).)

V! Design a program that results in the same path but with an oriented tool. Remember that the orientation at the end of a path segment must be the same as at the beginning.

Lab procedure:
Implement and test the program.

4.5.3 Programming in the tool-based coordinate system (VT5)

By aligning the tool and the path, it is possible (at least in some cases) to simplify the programming.

For example, one position can be used to find another relative position. Mathematically, this corresponds to the multiplication of two homogeneous matrices.

The following instructions are necessary:

```
;assume that XPref has the values
XPRef = {x 450, y -60, z 0, A 20, B 0, C 180}
;in the lab use the already corrected XPref from VT4
;but here the rotation of XPRef is shown with a rotation around the tool
;be careful, the tools orientation of y- and z-axis are reversed (C 180!!!).
Pref_new = XPRef : {x 0, y 0, z 0, A -20, B 0, C 0}

;now drive these points
PTP XPef
PTP Pef_new
```

First, the point P1 is approached. From there, relative commands are used. So the previous example is only nessasary to know the syntax used at P2 to rotate. You should move it once with PTP and once with LIN commands.

V! Design a program that results in the path described above by using this programming technique. Please consider acceleration, reduced velocity when necessary, reproduction of the path, and exact positioning in analogy to the previous tasks.

Lab procedure:
Implement and test the program.